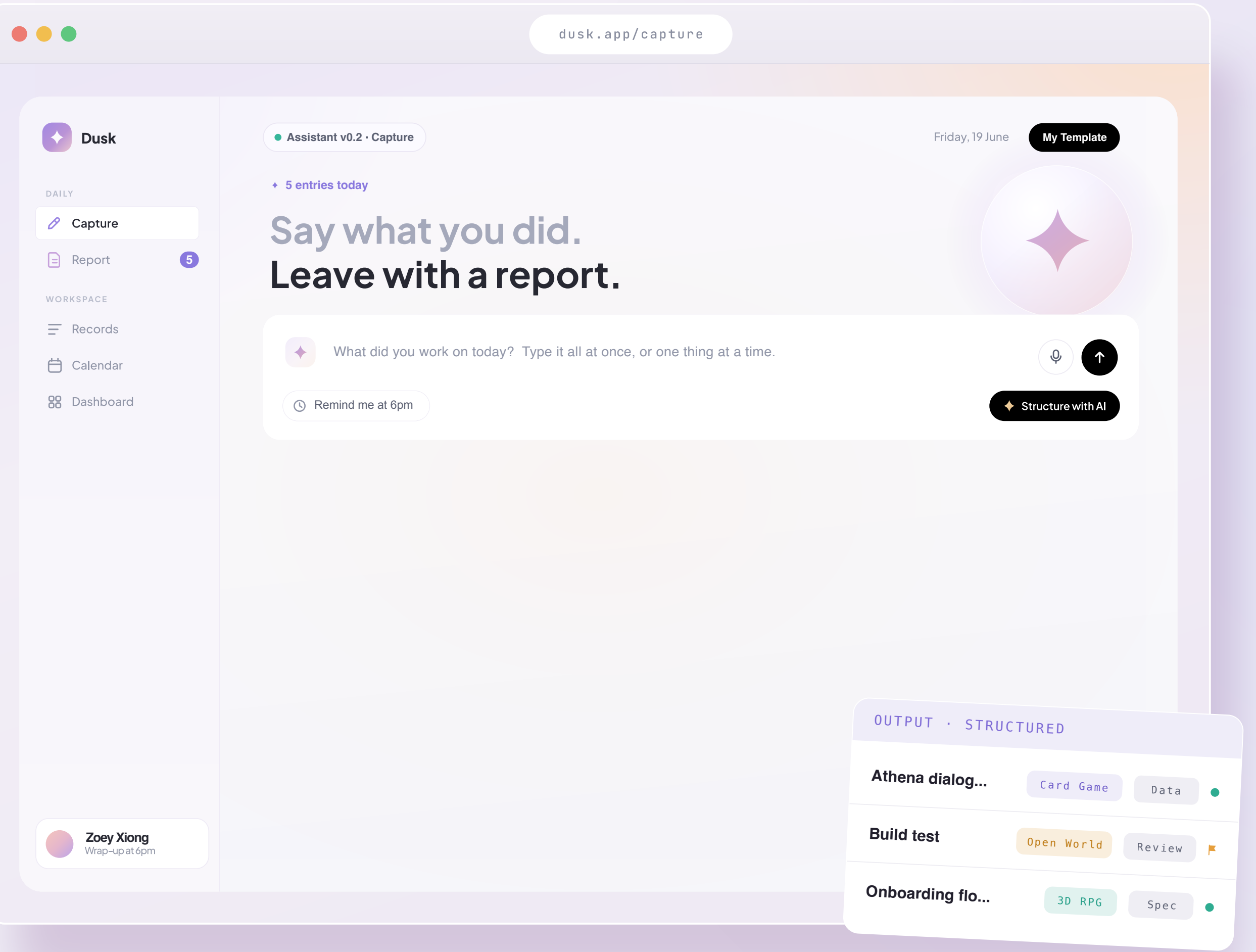
An AI daily wrap-up tool
speak your day, it files itself.

One sentence at the end of the day.
Dusk structures it into records and writes your reports from them.

Solo Project · Figma · Next.js · TypeScript · Groq API


dusk.app/capture
Dusk
DAILY
Capture
Report 5
WORKSPACE
Records
Calendar
Dashboard
Zoey Xiong
Wrap-up at 6pm
Assistant v0.2 · Capture
5 entries today
Friday, 19 June
My Template
Remind me at 6pm
Structure with AI
OUTPUT · STRUCTURED
Athena dialog… Card Game Data
Build test Open World Review
Onboarding flo… 3D RPG Spec

Say what you did.
Leave with a report.

What did you work on today? Type it all at once, or one thing at a time.

THE BRIEF

Your day, filed before you leave.

Most tools ask you to fill in a report. Dusk asks what you did, and writes it. The entries stay behind it, so the next report doesn't start from nothing.

THE RESULT

25 min → **7** min

The end-of-day report, before and after.

TIMELINE
MVP built in 2–3 days

STATUS
v0.2 · in development

PLATFORM
Responsive web app

AI FOCUS
Structured output prompting, Llama 3.3 70B

WHERE THIS STARTED

Reporting the work took longer than the work itself.

There was always another report due.

Daily, weekly, twice a month, monthly — four cycles running at the same time.

First I had to work out what the day had been.

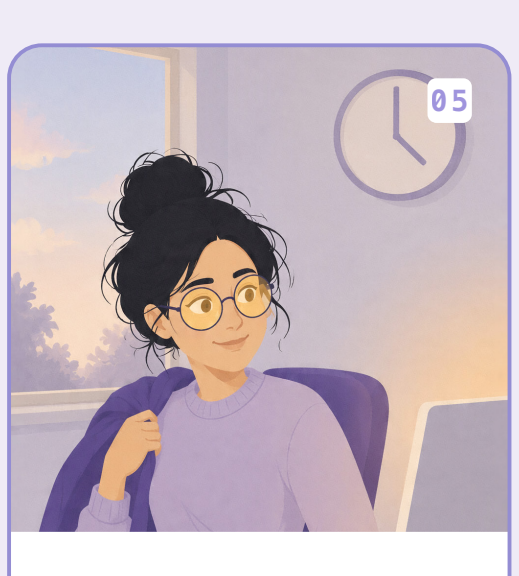
A day's work sat in Outlook, Slack, Jira and Confluence. I had to go through all four before writing anything.

So I started logging my work myself.

I kept it in the shape my reports needed, so whatever I had to write next, the record could already answer it.

The log didn't replace the report.

Copy from the log, paste into the model, wait, paste the result into the form. Every single time.



So I built the part I was doing by hand.

One place to type what I did, and the report comes back written. No copying, no pasting, no third window.

02 Card · In a survey of 12 people who report on their own work, each checked 3.3 tools on average before writing.

VALIDATING THE PROBLEM

The same routine, in twelve other jobs.

I built Dusk out of my own routine, so before rebuilding it I asked 12 people who report on their own work how they do it.

THEIR INDUSTRIES | Software engineering · Audit · Finance · QA · Product · Design · Lab · Film

12 of 12
All twelve report their work to someone. Nine of them on a fixed schedule.

6 of 12
keep a log they can sustain. The other six fell behind or stopped.

10 of 12
already use an AI tool to help write their reports.

> **One thing I had wrong.**
> I assumed everyone had my problem: remembering the day and digging it back out. Asked to rate that from 1 to 5, the group averaged 2.3. What they complained about instead was writing the same work up again for a different reader each time.

> "A lot of it has already been written in tickets or messages, so reporting means summarising the same information again."
> — Bruce · technical operations & QA

12 respondents · online questionnaire · August 2026, after v0.2 was built · recruited through my own network

THE PROBLEM I WANT TO SOLVE

The work only happens once.
The writing-up happens over and over.

Half of the survey respondents had already tried keeping a log and couldn't sustain it. But a vague record can't produce a usable report. Every time I made the input easier, the record got worse, and the three decisions below are how I settled it.

THREE DESIGN PRINCIPLES

01
Teach it your vocabulary once.
You fill in your projects and work types the first time you open Dusk. After that, "3D RPG, data config" is matched against those two lists, and anything unmatched waits for you to confirm.

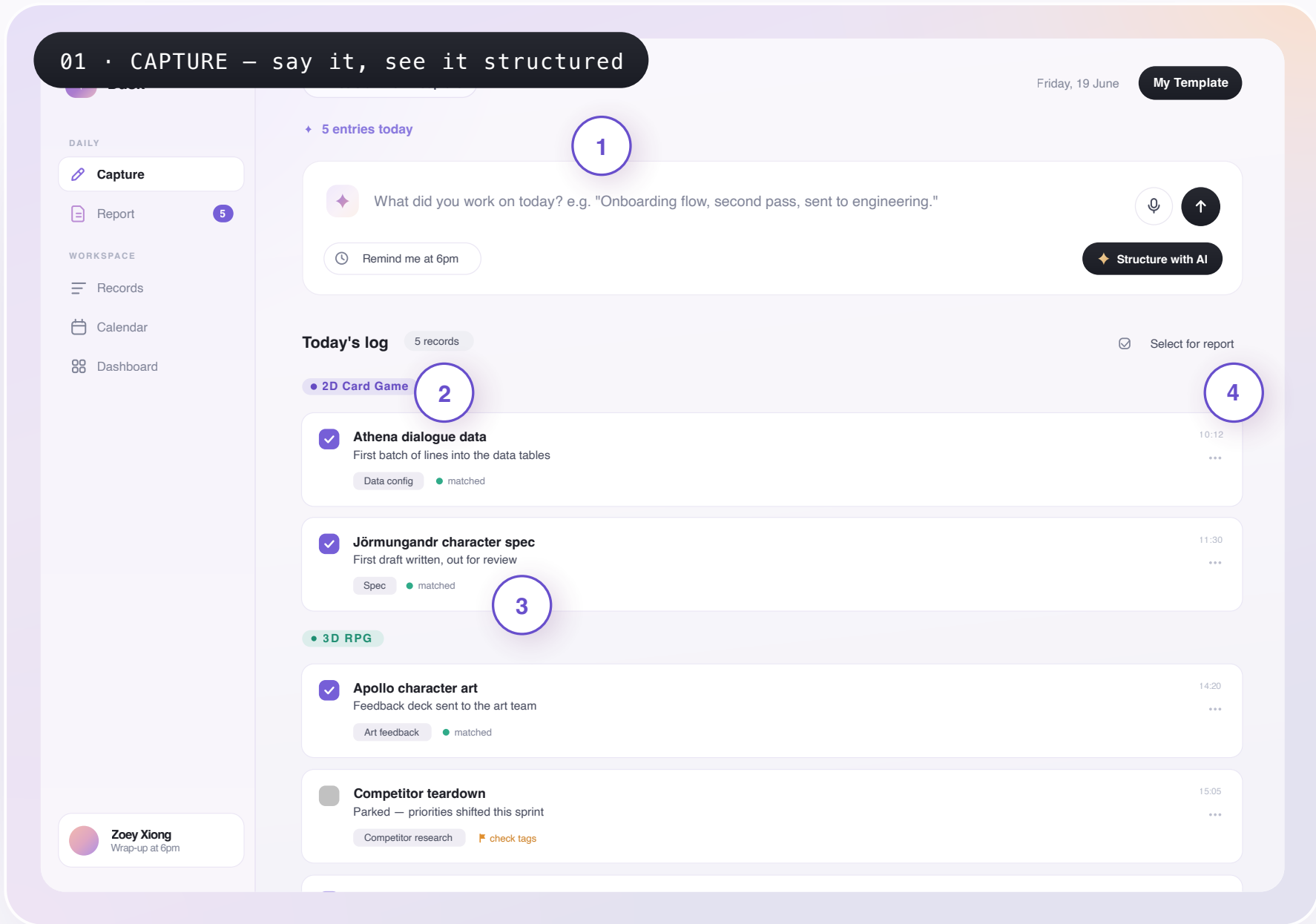
02
One record, three audiences.
You never write a report. You log what you did, and Dusk builds the report from those entries. The same set, formatted for whoever is reading it.

03
Nothing is filed until you say so.
A wrong entry is worse than none, because every report built on it inherits the mistake. Dusk drafts, you approve, and anything can be fixed in place before it lands.

THE FLOW

You type one sentence. Dusk turns it into records.

Brain–dump the day in one breath, Dusk sorts it into clean records by project and type, so the report writes itself.


01 · CAPTURE — say it, see it structured
Capture
Report
Records
Calendar
Dashboard
Zoey Xiong
5 entries today
Friday, 19 June
My Template
Remind me at 6pm
Structure with AI
Today's log
5 records
Select for report
2D Card Game
Athena dialogue data
Jörmungandr character spec
3D RPG
Apollo character art
Competitor teardown
1
2
3
4

What did you work on today? e.g. "Onboarding flow, second pass, sent to engineering."

1. **You type it however you'd say it**
   No fields,
   no format to learn.
2. **Structured by project & type**
   Matched against the two lists
   you set up.
3. **It tells you when it isn't sure**
   confirmed when it matched,
   to confirm when it didn't. ⚑
4. **You pick what's reported**
   Select the records that
   belong in today's report.

DESIGN DECISIONS

Three decisions, and what each one ruled out.

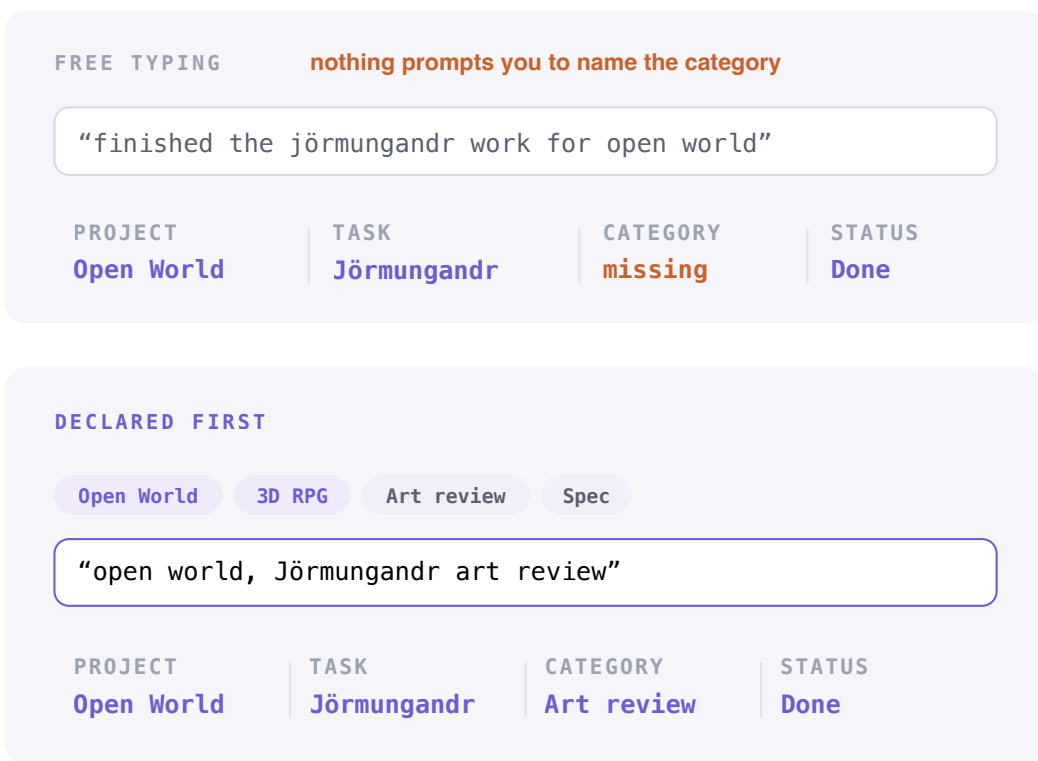
01

Set the vocabulary up first

BUILT IT FIRST
I built it as free typing first: you type anything, the model works out the rest.

WHY NOT
Every sentence would have to carry the project, task, category and status, the typing I wanted the tool to remove. And the longer an entry ran, the more the model dropped.

> WHAT I DID
> You declare your projects and work types once, on first use. After that, "open world, Jörmungandr art review" is enough, and every entry stays searchable by category.



FREE TYPING — nothing prompts you to name the category

"finished the jörmungandr work for open world"

PROJECT	TASK	CATEGORY	STATUS
Open World	Jörmungandr	missing	Done

DECLARED FIRST — Open World · 3D RPG · Art review · Spec

"open world, Jörmungandr art review"

PROJECT	TASK	CATEGORY	STATUS
Open World	Jörmungandr	Art review	Done

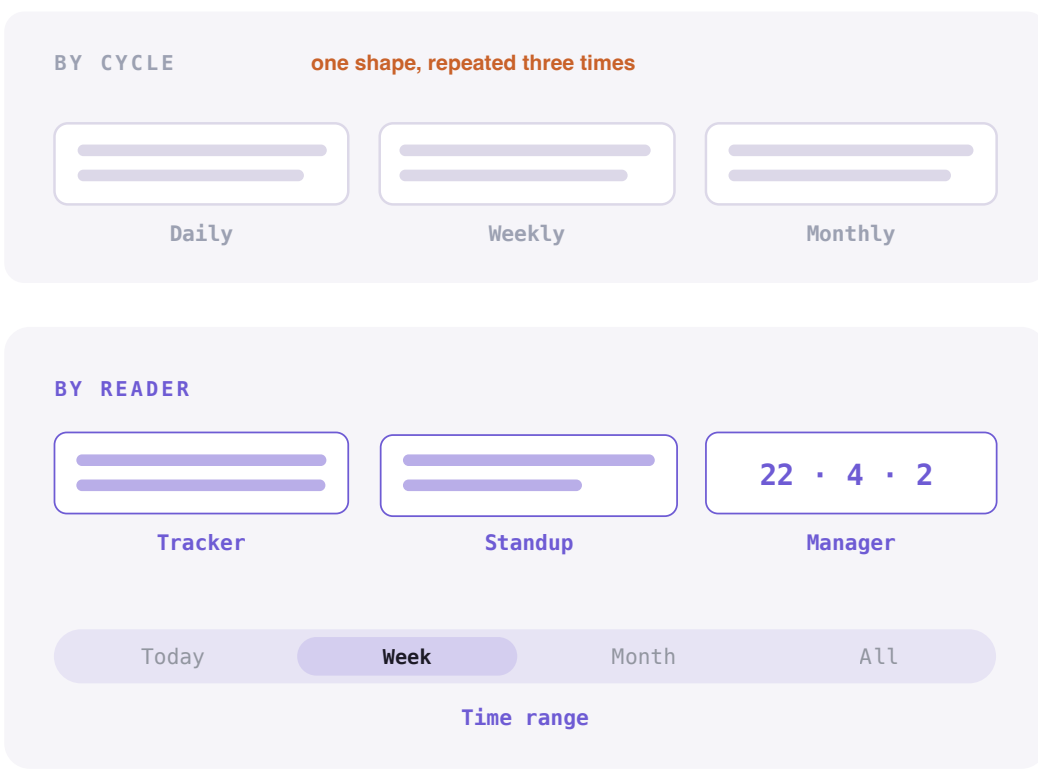
02

Split by reader, not by calendar

BUILT IT FIRST
I built it around the reporting cycles first — a daily, weekly and monthly page.

WHY NOT
Six months of my own sheet showed all three drew on the same entries only the emphasis changed. Three views were doing one job.

> WHAT I DID
> Split by what the report is for: a stand–up for the team, a 1:1 with your manager, and the full record. Time range became a separate filter, so one record has three uses.


BY CYCLE
one shape, repeated three times
Daily
Weekly
Monthly
BY READER
22 · 4 · 2
Tracker
Standup
Manager
Today
Week
Month
All
Time range

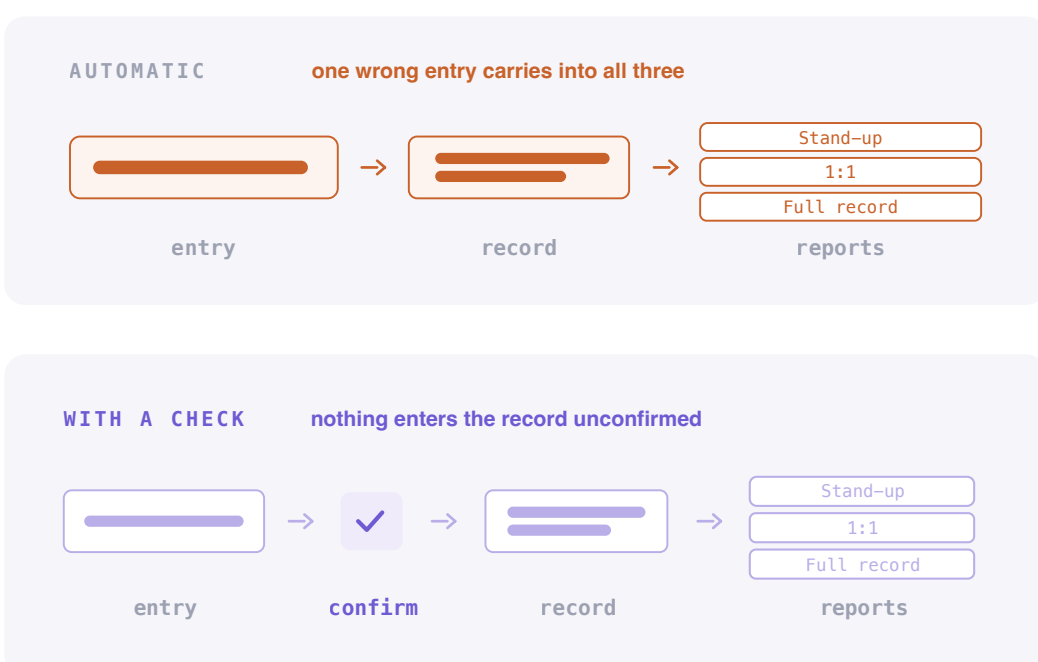
03

Keep a check in the way

BUILT IT FIRST
I built the MVP to file everything automatically, whatever you typed went straight into records.

WHY NOT
In testing, some entries came back under the wrong category or project. If the record is wrong, every report built from it inherits the error.

> WHAT I DID
> Dusk drafts, you confirm. Anything in the draft is editable, and only confirmed entries are filed.


AUTOMATIC
one wrong entry carries into all three
entry
record
Stand-up
1:1
Full record
reports
WITH A CHECK
nothing enters the record unconfirmed
entry
confirm
record
Stand-up
1:1
Full record
reports

STAYING IN THE LOOP

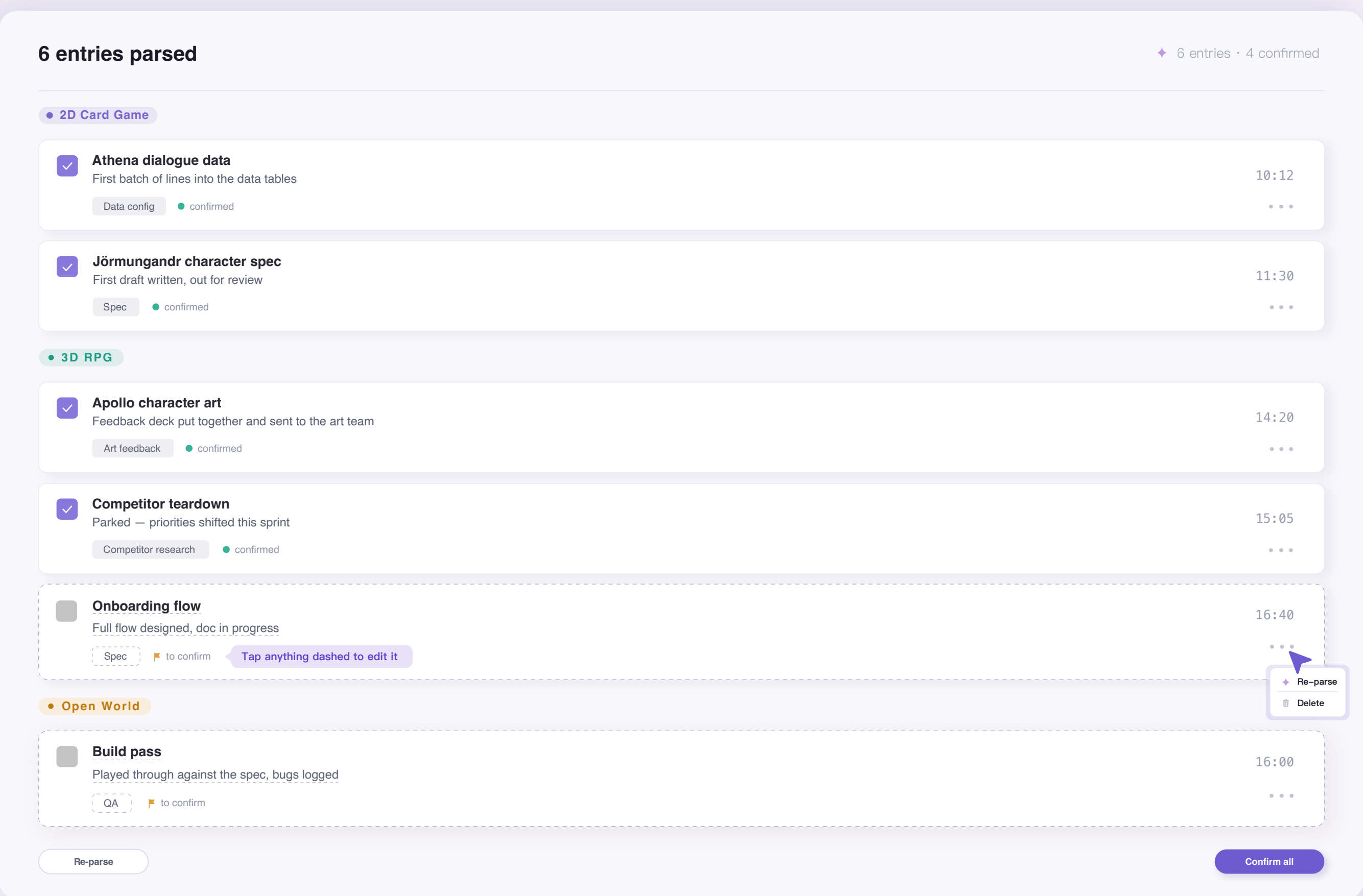
Dusk drafts. You confirm.

Adding a confirmation step was the decision. The design problem was making sure it doesn't turn into a new chore every evening.

01

You check the draft before it becomes a record.

After a parse, the model returns each item as a structured entry: task, project, type, status. Anything dashed can be changed right here, without leaving the screen.



6 entries parsed ✦ 6 entries · 4 confirmed

● 2D Card Game

☑ Athena dialogue data — First batch of lines into the data tables — Data config · ● confirmed — 10:12

☑ Jörmungandr character spec — First draft written, out for review — Spec · ● confirmed — 11:30

● 3D RPG

☑ Apollo character art — Feedback deck put together and sent to the art team — Art feedback · ● confirmed — 14:20

☑ Competitor teardown — Parked — priorities shifted this sprint — Competitor research · ● confirmed — 15:05

☐ Onboarding flow — Full flow designed, doc in progress — Spec · ⚑ to confirm — Tap anything dashed to edit it — 16:40

● Open World

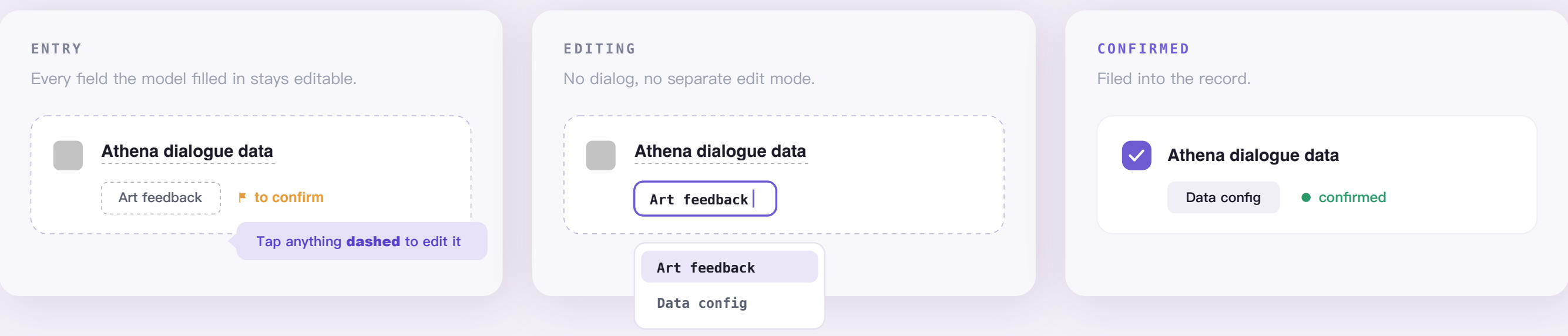
☐ Build pass — Played through against the spec, bugs logged — QA · ⚑ to confirm — 16:00

Re-parse
Delete
Re-parse
Confirm all

02

Fixing it happens where you found it.

If correcting one entry costs an extra click, correcting ten costs ten. Fixing something has to take less effort than letting it through.



ENTRY
Every field the model filled in stays editable.
☐ Athena dialogue data — Art feedback · ⚑ to confirm
Tap anything **dashed** to edit it

EDITING
No dialog, no separate edit mode.
☐ Athena dialogue data — Art feedback|
Art feedback
Data config

CONFIRMED
Filed into the record.
☑ Athena dialogue data — Data config · ● confirmed

03

The mistakes clustered in one field.

In my own use, a single entry often touches two categories at once, and which one wins is a team convention, not something the words can tell you. When I later asked 12 people what this tool would get wrong, half of them named the same field before seeing any of this.

Numbers, or a meeting?

INPUT
"Went over the battle damage formula with the lead engineer"

AI OUTPUT ~~Systems & numbers~~
ACTUALLY Meeting

The work was about numbers, but the task itself was a meeting. Both are categories in my list, and the model had to pick one.

Research, or a document?

INPUT
"Pulled apart a competitor's monetisation system and wrote it up"

AI OUTPUT Spec
ACTUALLY Competitor research

The research was the work; the document was the output. The model caught the last and most concrete thing in the sentence.

> Both times the model read the sentence correctly. It just had two valid answers and no way to choose between them. There will always be entries like these, so the fix has to be quick: change the category where it sits, and confirm.

THE DATASET

Where the data came from.

Six months of logging my own work in a real job.

`work-log.xlsx` · 242 rows · 3 projects

No.	Task	Priority	Week	Quarter	Completion Date	Status	Type	Project	Notes
1	Monster art assets — production tracking	High	11.24 - 11.28	Q4	2025/11/25	Done	Spec	2D Card Game	Reference art updated for the final boss
9	Monster attribute data — compiled	Medium		Q4		On Hold	Spec	2D Card Game	Monster trait comparison sheet
18	09 character combat data — configured	High	11.24 - 11.28	Q4	2025/11/28	Done	Data Config	Open World	Interaction table configured
42	1004 base character art — feedback	High	12.22 - 12.26	Q4	2025/12/24	Done	Art Feedback	2D Card Game	Feedback deck sent
78	Build pass — combat balance	Medium	02.23 - 02.27	Q1	2026/02/25	Done	QA Test	Open World	Played against spec, bugs logged
151	Weekly meeting notes — written up	High	04.27 - 05.01	Q2	2026/04/27	Done	Meeting	3D RPG	
209	17 character data — configured	High	05.18 - 05.22	Q2	2026/05/20	Done	Data Config	Open World	Interaction / option entry tool
242	1019 character profile — spec	Low		Q2		On Hold	Spec	2D Card Game	Tier 2 characters deferred, tier 1 first

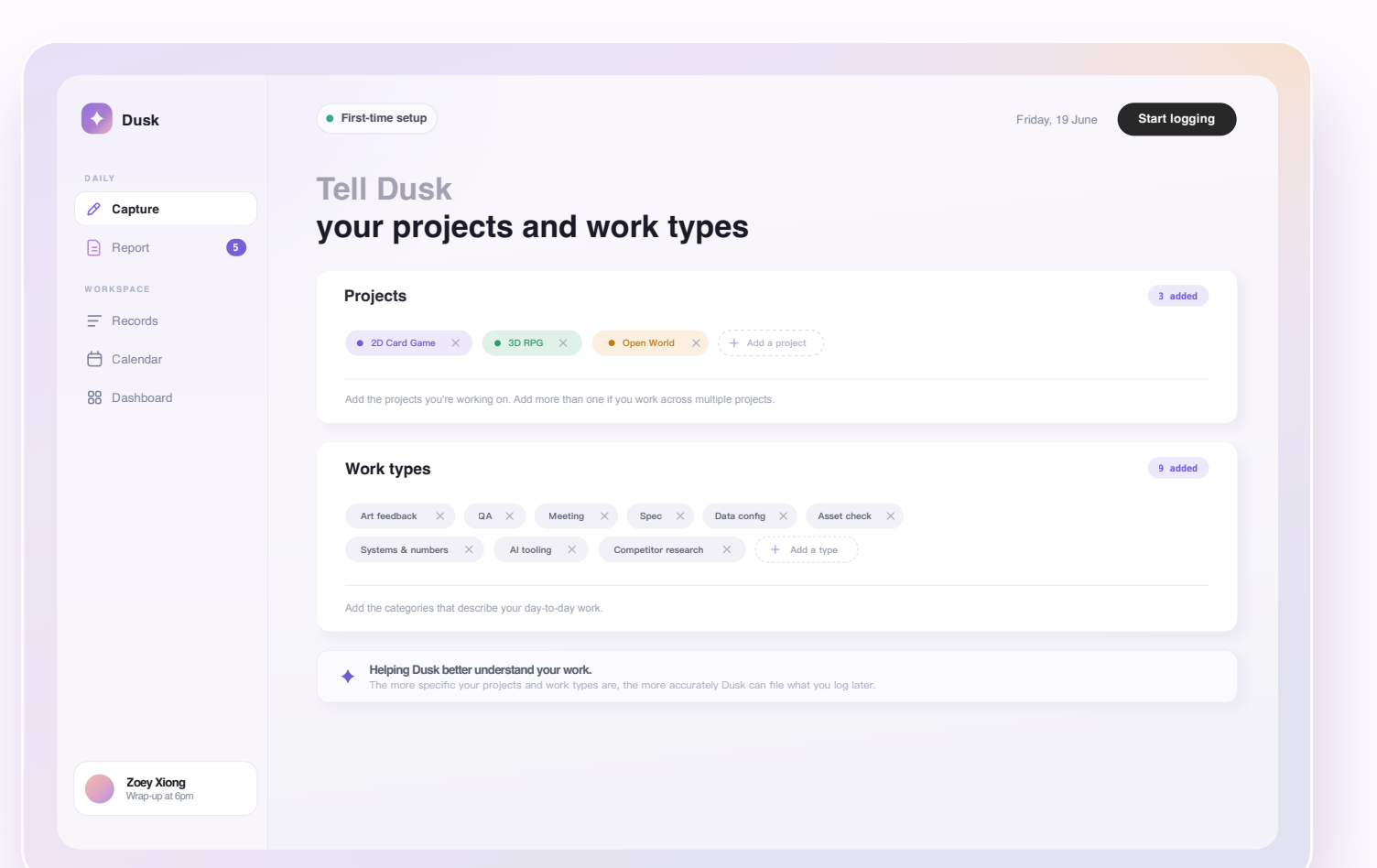
the real log, 8 of 242 rows

Task, project, type and status — I kept adjusting these columns against real reporting until the set could carry every format I needed. That's the shape Dusk parses into.

PROMPT & OUTPUT

How I got a language model to do this reliably.

Two API calls do all the AI work: one turns what you typed into structured entries, the other turns those entries into a report.



Tell Dusk
your projects and work types

Projects — 3 added
2D Card Game · 3D RPG · Open World · Add a project
Add the projects you're working on. Add more than one if you work across multiple projects.

Work types — 9 added
Art feedback · QA · Meeting · Spec · Data config · Asset check · Systems & numbers · AI tooling · Competitor research · Add a type
Add the categories that describe your day-to-day work.

Helping Dusk better understand your work.
The more specific your projects and work types are, the more accurately Dusk can file what you log later.

Zoey Xiong
Wrap-up at 6pm

01 WHAT YOU FILL IN

You give it the vocabulary before it reads anything.

Before using Dusk for the first time, you provide your projects and work types. This helps the model understand what your work involves, allowing it to match short entries to the right project and category later on.

02

Your two lists are sent with every request.

Both lists are sent with every request, alongside the text and the date.

POST /api/parse-entry

```
{
"text"  : "finished 80% of the character spec for 2D Card Game,
joined the QA sync, chased a config bug —
fixing it tomorrow"           ,
"projects"  : [  "2D Card Game", "3D RPG", "Open World"],
"task_types"  : ["Spec", "Meeting", "Data config", "Art review"],
"date"  : "2026-05-11"
}
```

The model is not trained on your lists.
They are sent as two arrays with every request. Change your projects in settings and the next parse uses the new list.

Today's date is passed in.
A model has no idea what day it is. “tomorrow” and “Friday” only resolve because the current date travels with the text.

When nothing matches, it says so.
The rule is to fall back to Uncategorised rather than invent a label. A blank I have to fill beats a plausible category that's wrong.

03

What the prompt actually asks for.

Seven rules, and a fixed structure for what comes back.

1. Each distinct piece of work becomes its own entry.
2. Match the project against the supplied list. If nothing matches, use “Uncategorised”.
3. Match the work type against the supplied task–type list.
4. “finished / done” → Done.   “working on / looking into” → In progress.
5. Infer priority from context. Default to Medium with no clear signal.
6. Convert “tomorrow” / “Friday” / “15 May” to an absolute date using the date supplied.
7. Return only the JSON array. No other text, no markdown.

Return Format

```
[{
"name"  : "…",
"project"  : "…",
"task_type"  : "…",
"status"  : "   Done|In progress|On hold",
"priority"  : " High|Medium|Low",
"date" : "YYYY-MM-DD or null"
}]
```

04

The second call is the only one that changes.

The parse prompt never changes. The second call is the one that writes the report, and its instructions change depending on who is going to read it.

Tracker
for the record

`No prose. Return the entries in the log's own column order.`

`task, project, type, status, date. One row each, nothing merged or summarised.`

Standup
for the team sync

`Cover what was done yesterday and what's planned for today.`

`Keep each item concrete enough for someone else to know where things stand. Flag anything blocking another person.`

Manager
for the 1:1

`Lead with the totals: how many of each work type, across which projects.`

`Then the deliverables that landed, then blockers and anything needing a decision. Covers a longer span than a single day.`

05

Three things I had to solve in code.

The prompt gets you most of the way. These three sit in the application, not in the instructions.

```
how much the model may vary its answer

parse:    0.3   // same fields every time
report:   0.7   // should read like a person
tracker:  0.2   // a list, not writing
```

The same model, tuned differently for parsing and for writing.
Temperature controls how much a model is allowed to vary its answer. Pulling fields out of a sentence should give the same result every time, so that call runs low. Writing a report should read like a person wrote it, so that one runs higher.

```
the model wraps JSON in a code block, the parser breaks

// strip the fence before parsing
responseText.replace(/```json|```/g, '')
```

The model doesn't always return clean JSON.
The prompt says to return the array and nothing else, and the model mostly listens. When it wraps the JSON in a code block instead, this line strips it before parsing.

```
what a model swap actually costs

2  files call the model
3  vendors tried
1  JSON shape everything else depends on
```

I changed the model twice without touching the product.
I wrote the prompts for Claude, tried Google's, and shipped on Groq's Llama 3.3. Only two files call the model. Everything else depends on the JSON shape, not on which vendor produced it, so each swap was a config change.

DESIGN SYSTEM

Nine colours. Eight sizes.

Every colour, size and surface in Dusk follows the same set of rules.

Text
Four shades, one for each level of importance.

#242334	#343541	#6D7382	#A1A6B6
ink · headings	ink-soft · icons	text · body	text-muted · meta

Accent and status
Each colour is tied to a specific action or state in the interface.

#8174DA	#9A8EE4	#D6D1E7	#2D9A6A	#E59E3B
accent · selected	accent-soft · marks	accent-tint · chip	positive · confirmed	attention · to confirm

Backgrounds
Page, card, tag and border — each layer of the interface has its own shade.

#FFFFFF	#F3F1F7	#CECBD8	#DDE3F8	#EAE4F9 → #FBE6DB
surface · cards	subtle · tag fill	border · dividers	canvas · page	dusk · app background

Type scale
Eight steps. One size per role, on every screen.

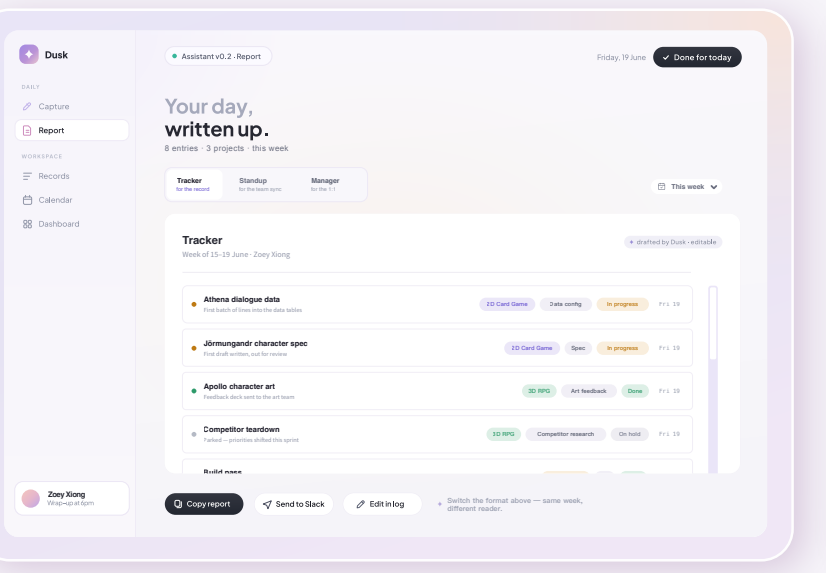
Say what you did.	32 / 700	Hero
Your day, written up.	24 / 700	Screen title
Today's log	20 / 600	Section
What did you work on today?	16 / 400	Input field
Athena dialogue data	14 / 600	Record title
First batch of lines into the data tables	13 / 400	Description
Data config · confirmed · 10:12	12 / mono	Tags, time, counts
DAILY · WORKSPACE	11 / mono	Sidebar labels

THE ENGINE

The same work. Read three different ways.

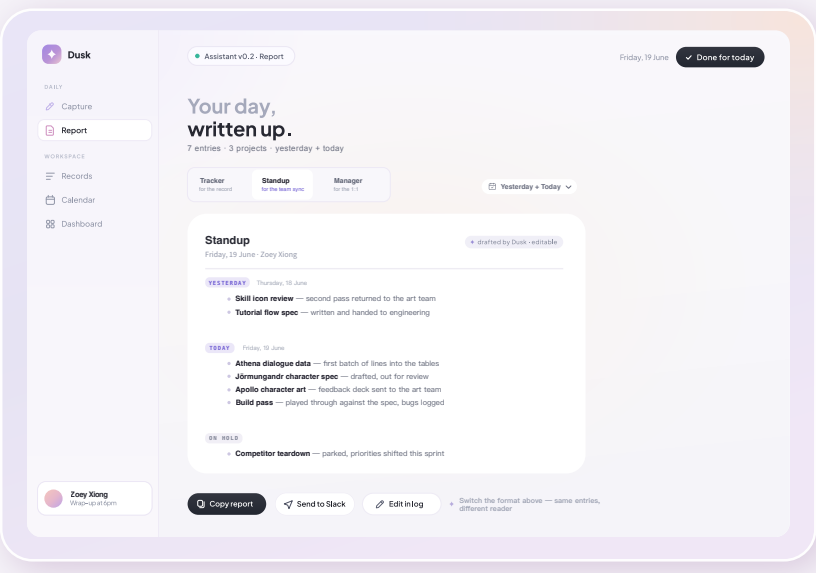
Friday, 19 June — the same eight records, shown as three different reports.

input · 1 log   8 records   → 3 formats



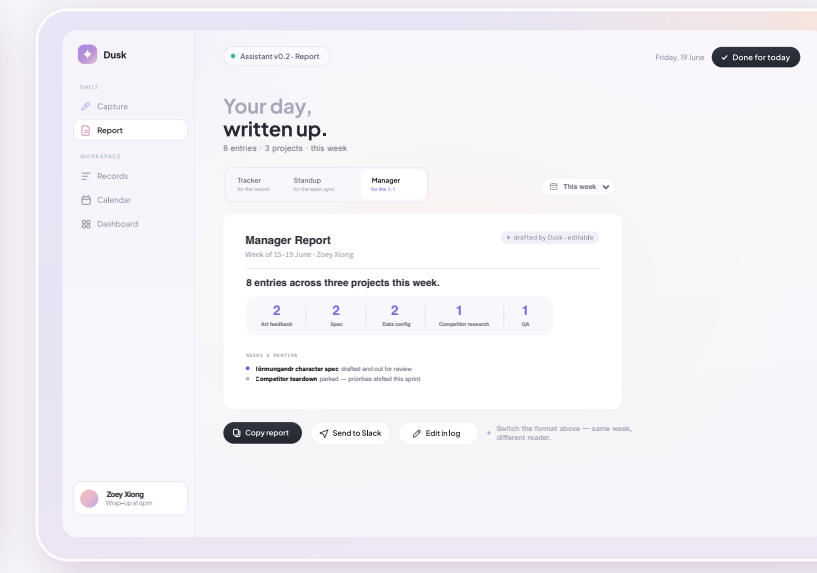
Tracker — for the record

The full log for any period I need to look back on.



Standup — for the team sync

Yesterday and today, quick enough to sync the team in a minute.



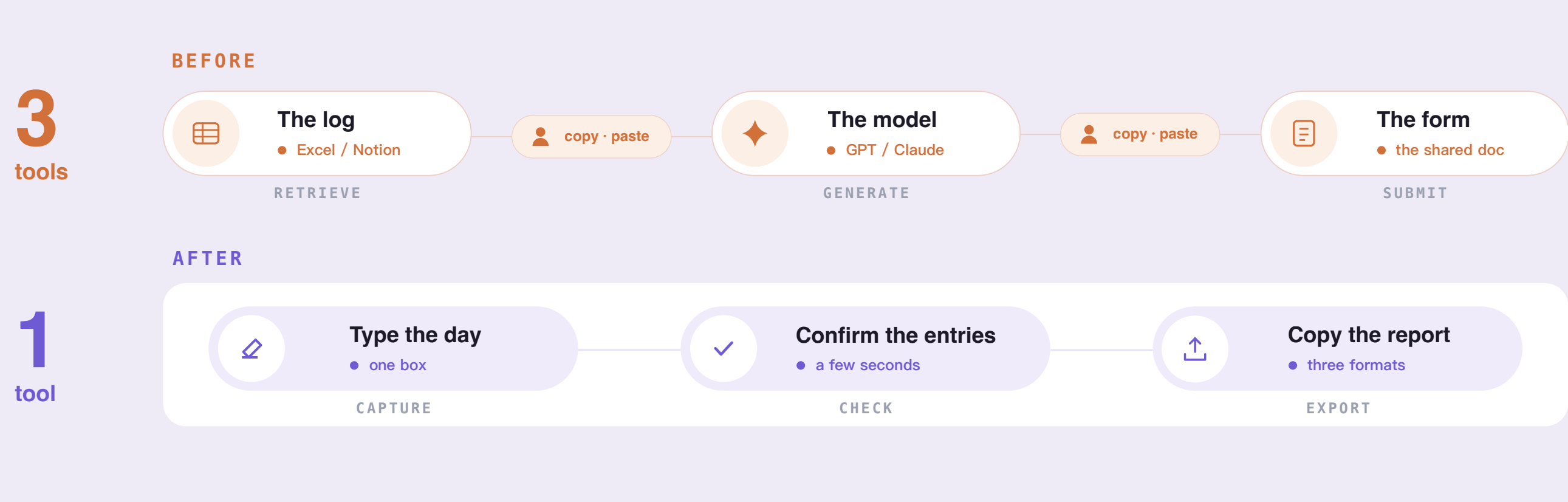
Manager — for the 1:1

Recent work counted up, ready to take into a 1:1.

THE WORKFLOW

Three tools became one.

The end of my day used to run through a log, a chat window and a submission form. Now it runs through one box and one confirmation.

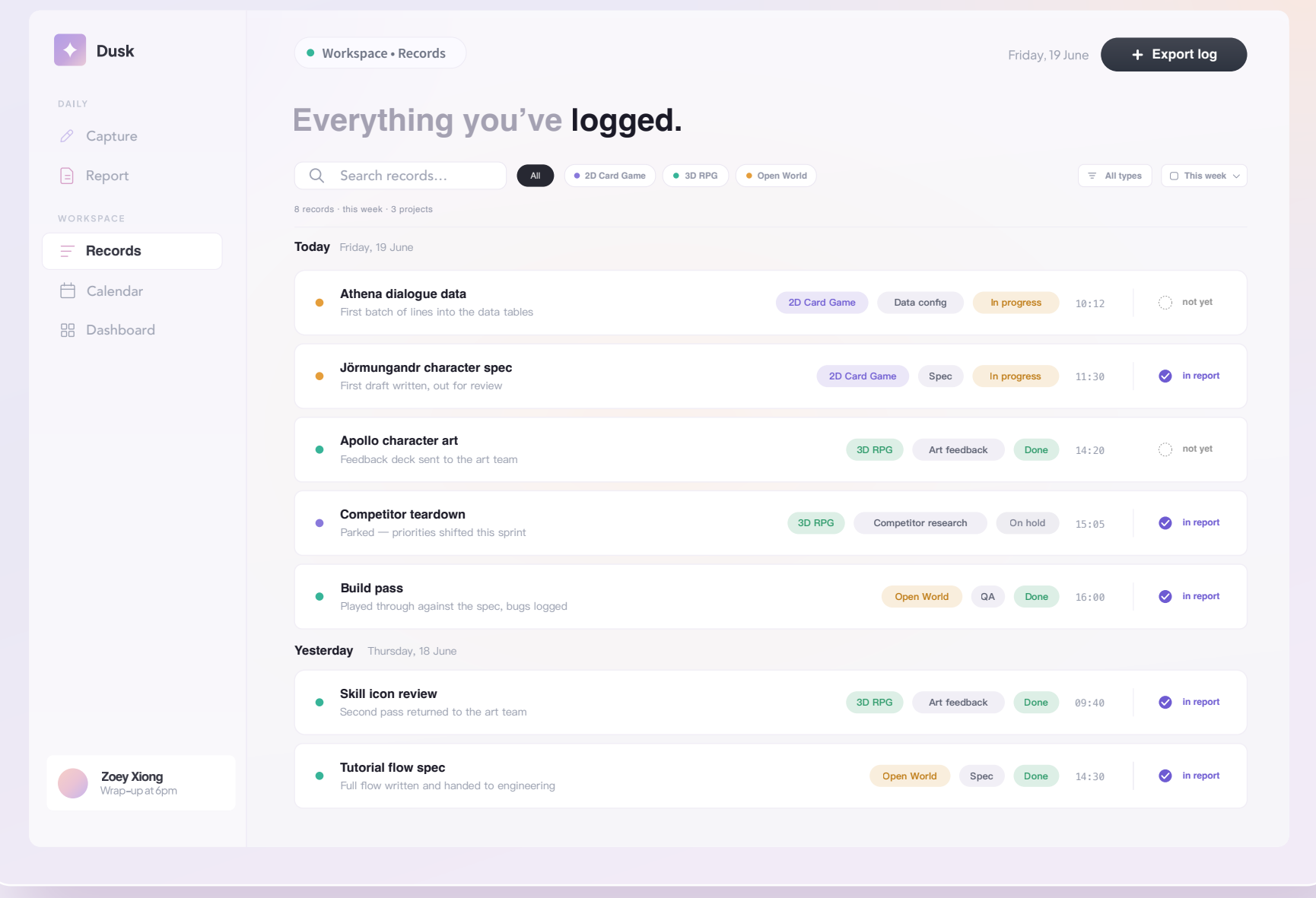


THE WORKSPACE

The report is what I send. The record is what I keep.

A report gets pasted somewhere and forgotten. The entries behind it are what I go back to, so the workspace is built for finding things again.

> "What did I ship last quarter?"
> "How many art reviews in July?"
> "What was I doing in March?"



Everything you've **logged.**

8 records · this week · 3 projects

Today · Friday, 19 June

Record	Project	Type	Status	Time	Report
Athena dialogue data First batch of lines into the data tables	2D Card Game	Data config	In progress	10:12	not yet
Jörmungandr character spec First draft written, out for review	2D Card Game	Spec	In progress	11:30	in report
Apollo character art Feedback deck sent to the art team	3D RPG	Art feedback	Done	14:20	not yet
Competitor teardown Parked — priorities shifted this sprint	3D RPG	Competitor research	On hold	15:05	in report
Build pass Played through against the spec, bugs logged	Open World	QA	Done	16:00	in report

Yesterday · Thursday, 18 June

Record	Project	Type	Status	Time	Report
Skill icon review Second pass returned to the art team	3D RPG	Art feedback	Done	09:40	in report
Tutorial flow spec Full flow written and handed to engineering	Open World	Spec	Done	14:30	in report

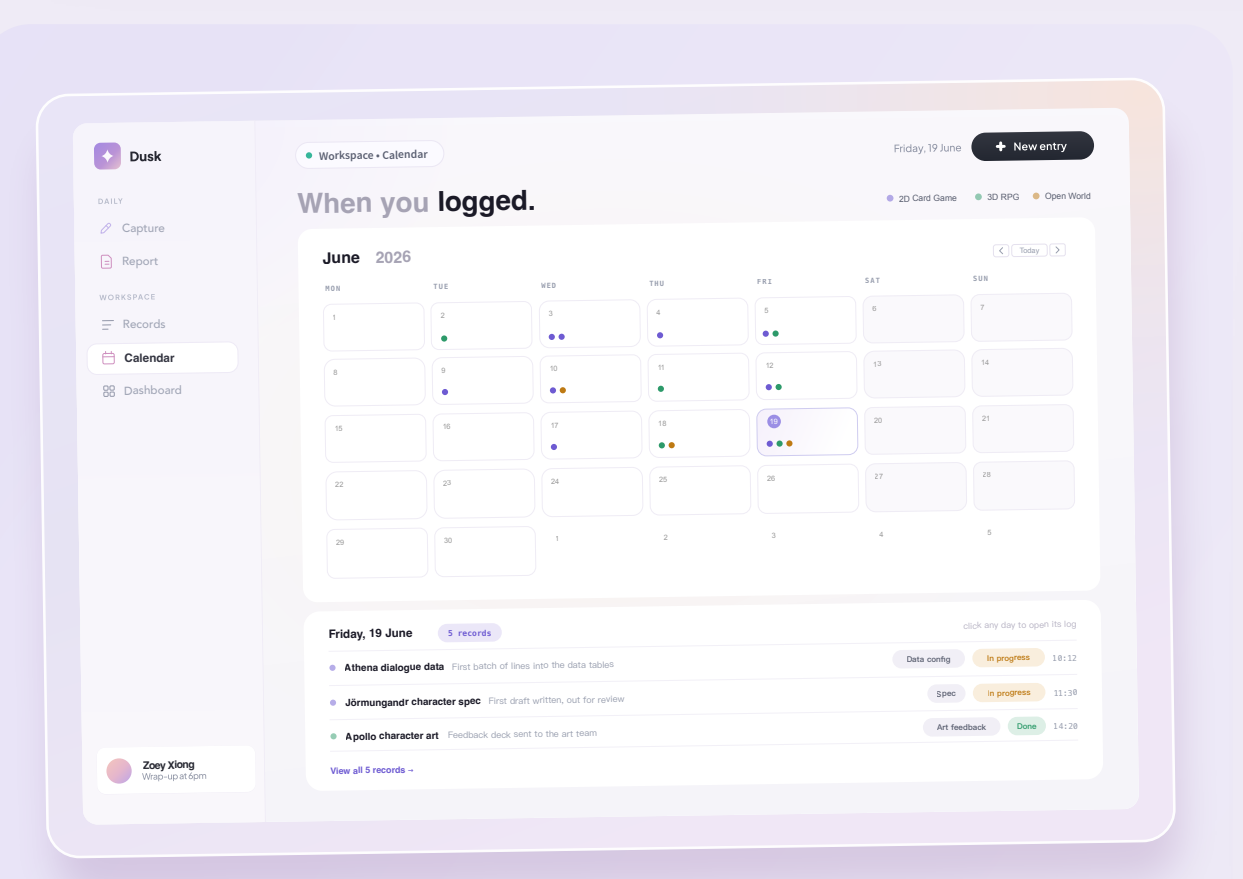
01

Records

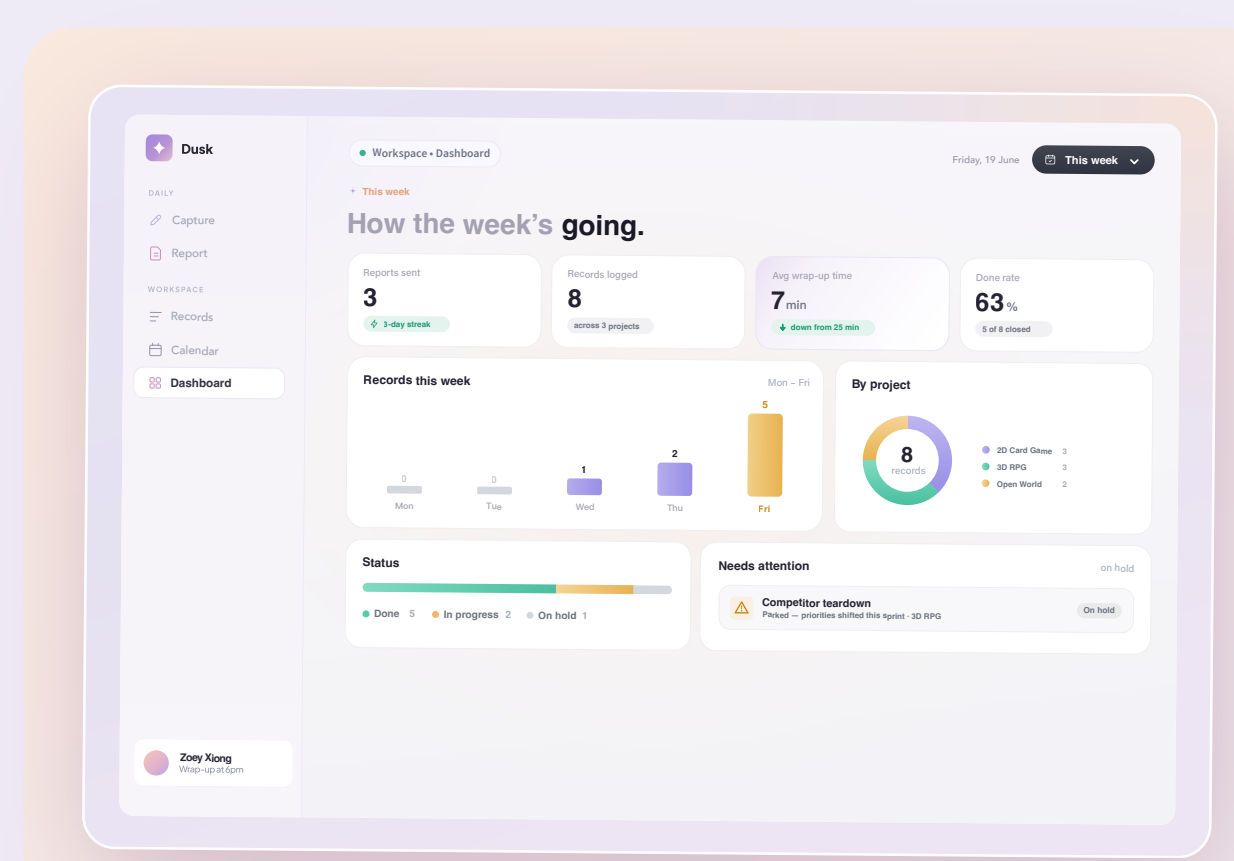
Every entry, searchable

The full log, filtered by project, work type or date. It answers the questions above, and it's why I started keeping a log at all.

242	records over six months
9	work types, declared once
3	filters — project, type, date



Calendar — June 2026



Dashboard — This week

02

Calendar

A month at a glance, one dot per project per day. You can see which weeks were heavy, which projects took over, and which days have nothing on them at all.

18	days logged this month
5	records logged on the busiest day

03

Dashboard

Pick any range and Dusk totals it up — how much work, split by project and by status, and how long the wrap-up takes. You log; the counting happens on its own.

7 min	avg wrap-up, down from 25
63%	done · 5 of 8 closed

TESTING THE PRODUCT

I showed the screens to the same twelve people from the survey.

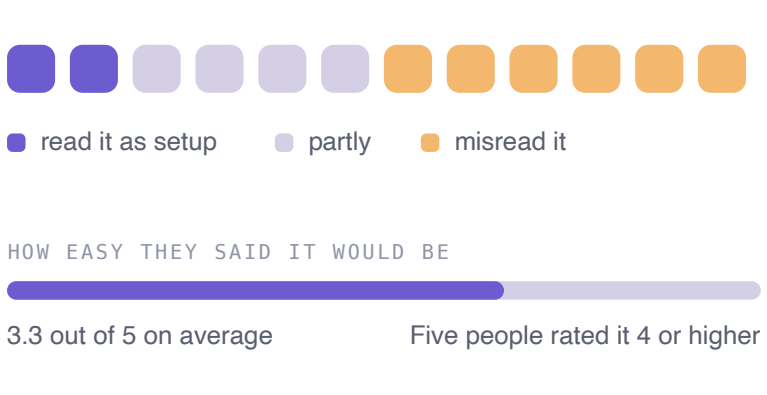
None of them had used Dusk before, and I gave no explanation of what any screen was for.

01

Only two of twelve read the setup screen correctly.

I showed them the first-time setup screen with no explanation, and two described it as a one-time setup. Six read it as the daily logging screen, or as planning work they hadn't done yet.

> They found it easy to understand and filled it in wrong. That gap is the finding.

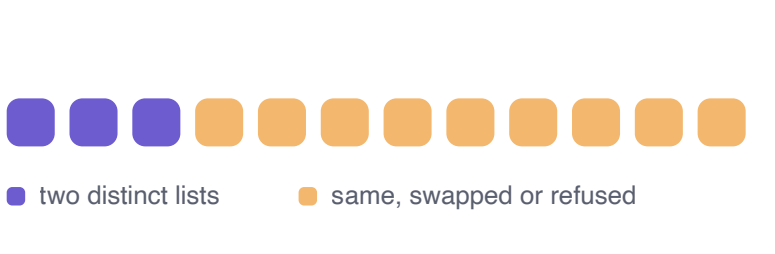


02

People don't naturally split their work into two separate lists.

I asked them to type what they would put in Projects and in Work types. Two wrote the same thing in both, one swapped them, one wrote a job title, and one refused.

> The pattern assumes everyone holds a stable two-axis model of their own work. But most don't.



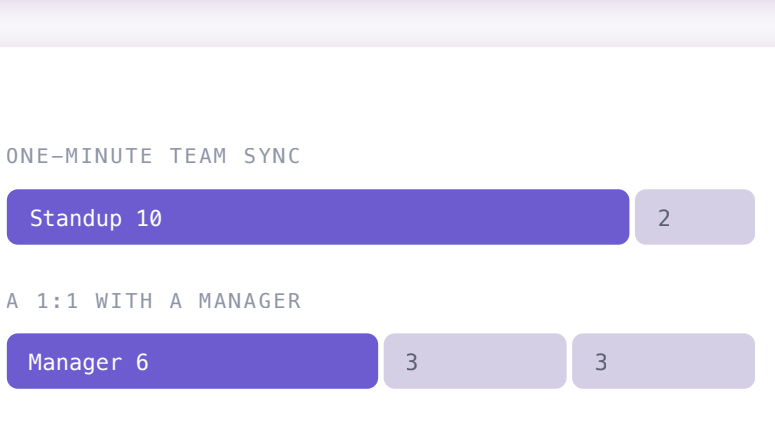
“Don’t think I’d use it because each project contains many work types.”
— Claire, product designer

03

Most testers picked Standup for the team sync. Tracker lost the role I designed it for.

I showed the three report screens with their names hidden, and asked which one they would use in three situations: a stand–up with the team, a 1:1 with manager, and a record to look back on in six months.

> The split by reader holds. But people would keep Manager, not Tracker.

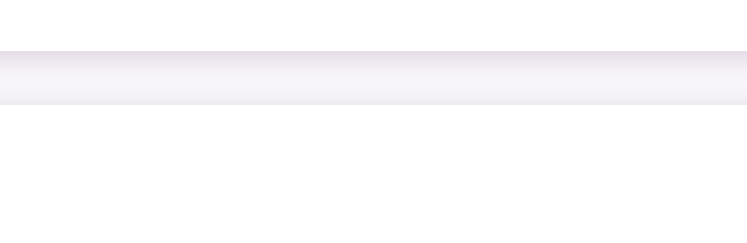


04

Nobody wanted to remove the confirmation step when logging their work.

Six people called the confirmation step necessary and said they would want to check what was filed. The other six said it was fine for a few records, but that a large batch would become a burden.

> The decision was right. I still need to solve it when there are too many records.



0 people would remove the step

12 respondents · static screens in an online questionnaire · August 2026

WHAT I'D DO NEXT

Three things the testing left open.

01

The setup screen

Ten of twelve misread it. The two lists need to come out of a few real sentences instead of a blank form.

02

Confirming one record at a time

Nobody wanted the step gone, but six said a large batch would be a burden. Confirm in bulk, or only the ones the model is unsure about.

03

What Tracker is for

The tracker was built for long-term reference, but most people chose the Manager summary instead. It needs to be tested against the use case it was designed for.

Dusk

An AI wrap-up tool for the end of the working day.

Designed & built end-to-end by Zoey Xiong

PRODUCT DESIGN · FRONT-END · LLM INTEGRATION · 2026